Team Name : The Slaying Ninjas

Team Leader : Padma Priya T R

Team Members : Padmesh G N, Sharan B

Team Mentor : Mrs. V. Thanammal Indu

Problem Statement : Online Fake Content Identification|

Datase : https://raw.githubusercontent.com/laxmimerit/fake-real-news-dataset/main/data/True.csv

```
from keras.models import Sequential
```

+ Code + Text

```
# libriaries used are loaded

from google.colab import drive
import pandas as pd
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from pprint import pprint
from __future__ import absolute_import, division, print_function, unicode_literals
import collections
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.compat.v1.keras.layers import CuDNNGRU
import tensorflow as tf
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Dense, Dropout, Embedding, GRU, LSTM, RNN, Spatia
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
print(stopwords.words('english'))
from nltk.tokenize import word_tokenize
from tensorflow.keras.callbacks import EarlyStopping
import pickle
from tensorflow.keras.layers import BatchNormalization
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', "you're", "you'v
```

```
from tensorflow.keras.layers import BatchNormalization
```

```
df_test=pd.read_csv("https://raw.githubusercontent.com/avalmahsa/Fake-news-detection/ma
```

```
## Import dataset
df1 = pd.read_csv('https://raw.githubusercontent.com/laxmimerit/fake-real-news-datase
df1['label']=int(1)
df2= pd.read_csv('https://raw.githubusercontent.com/laxmimerit/fake-real-news-dataset
df2['label']=int(0)
df3=pd.read_csv("https://raw.githubusercontent.com/avalmahsa/Fake-news-detection/main
df1=df1.drop(axis=1, index=None, columns="subject")
df2=df2.drop(axis=1, index=None, columns="subject")
df1=df1.drop(axis=1, index=None, columns="date")
df2=df2.drop(axis=1, index=None, columns="date")
df1["text"]=df1["title"]+" "+df1["text"]
df2["text"]=df2["title"]+" "+df2["text"]
df1=df1.drop(axis=1, index=None, columns="title")
df2=df2.drop(axis=1, index=None, columns="title")
df_train=pd.concat([df1,df2,df3]).reset_index(drop=True)

```

```
df_train=df_train.drop(axis=1, index=None, columns="id")
```

```
#df_test
```

```
x =df_train.text
y =df_train.label
x_train, x_valid, y_train, y_valid = train_test_split(x, y, test_size=0.2)
```

```
# maximum number of words from the resulting tokenized data which are to be used
vocab_size = 40000
max_len = 100
# build vocabulary from training set
tokenizer = Tokenizer(num_words=vocab_size)
# upadting the internal vocalulary based on the list of text, so it creates the vocabul
# index based on word frequnecy so every word gets a unique interger value so lower int
# mean more frequent word.
tokenizer.fit_on_texts(x_train)
def _preprocess(list_of_text):
    # pads sequence to the same length (all sequences in a list to have the same length
    # does so by padding 0 in the beggining of each sequence until they have the same l
    # the longest sequence.
    return pad_sequences(
        # transforms each text in texts to a sequence of integers. It takes each word
        # in the text and replaces it with its corresponding integer value from the
        # dictionary.
        tokenizer.texts_to_sequences(list_of_text),
        # takes in the pre-defined input (40) as maximum length of all sequences.
        maxlen=max_len,
        # does padding after each sequence
        padding='post',
    )
# padding is done inside:
x_train = _preprocess(x_train)
x_valid = _preprocess(x_valid)
```

```
print(x_train.shape, y_train.shape)
print(x_valid.shape, y_valid.shape)
```

```
(83918, 100) (83918,)
(20980, 100) (20980,)
```

```
# Stop word removal before passing it to preprocessing
x =df_train.text
def filter_stop_words(x, stop_words):
    for i, sentence in enumerate(x):
        new_sent = [word for word in sentence.split() if word not in stop_words]
        x[i] = ' '.join(new_sent)
    return x
stop_words = set(stopwords.words("english"))
train_sentences = filter_stop_words(x, stop_words)
```

```
/usr/local/lib/python3.7/dist-packages/ipykernel_launcher.py:6: SettingWithCopyWarnir
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
```

```
# these match as we can see. It has 23 words and the rest is padded to match 40 which w
# predefined argument.
print(x_train[:4])
print(tokenizer.sequences_to_texts(x_train[:4]))
```

```
[[   16  3465 13771     9  4080  2590   281  1449   350  7972     6   849
      2    24   232  2340     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
      0     0     0     0]
 [  496   112    44     4  6906  3792  8077    54   761     3   197     1
    820  2182     3   155   158  2959    90     2  1352  5611 11972    86
  32075     5   440  1660  7090     2     1  8228     3 11519  2631    25
    647     4  8078  9199     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
      0     0     0     0]
 [  218     3    30 16089   792   240    15    44    30  2564  1242  4866
      1   240    15    67    49   199    85    45  5363   147 10211     3
      1   796     6    55    11   180 11790  2854 26487  1744   648     3
    765     5  2799     1  2803     7 19904     1 10477 23651  1224  4026
    146     8   290   250    15     4  2023  1242  4456   955     2    25
   1361     1   796  3007    67    25    46   331  1080  9854   453   146
      6   383   112   581     2     4  1146  1114   863     3  1242 12928
    184     6     1  1182   195   218     3     6   663   450   139    27
    386   496    45  1090]
 [  749 10277 13049    18    31   951     2   508     4  1235     3  2543
   5652   880     0     0     0     0     0     0     0     0     0     0
      0     0     0     0     0     0     0     0     0     0     0     0
```

```
       0     0     0     0     0     0     0     0     0     0     0     0
       0     0     0     0     0     0     0     0     0     0     0     0
       0     0     0     0     0     0     0     0     0     0     0     0
       0     0     0     0     0     0     0     0     0     0     0     0
       0     0     0     0     0     0     0     0     0     0     0     0
       0     0     0     0]]
['with industrial shelves on ceiling fan without putting little springs in seems to b
```

```
# CNN
EPOCHS = 8
MAX_SEQUENCE_LENGTH = 100
MAX_NB_WORDS = 200000
EMBEDDING_DIM = 100
VALIDATION_SPLIT = 0.2
embedding_matrix = np.random.random((len(train_sentences) + 1, EMBEDDING_DIM))
embedding_layer = Embedding(len(train_sentences) + 1,
                            EMBEDDING_DIM,
                            weights=[embedding_matrix],
                            input_length=MAX_SEQUENCE_LENGTH)
modell = Sequential()
modell.add(embedding_layer)
modell.add(Dropout(0.2))
modell.add(Conv1D(filters=32, kernel_size=5, padding='same', activation='relu'))
modell.add(MaxPooling1D(pool_size=2))
modell.add(Conv1D(filters=64, kernel_size=3, padding='same', activation='relu'))
modell.add(MaxPooling1D(pool_size=2))
modell.add(LSTM(100))
modell.add(BatchNormalization())
modell.add(Dense(256, activation='relu'))
modell.add(Dense(128, activation='relu'))
modell.add(Dense(64, activation='relu'))
modell.add(Dense(1, activation='sigmoid'))
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=Tru
modell.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
print(modell.summary())
modell.fit(x_train, y_train, epochs=EPOCHS, batch_size=64,validation_data=(x_valid,y_va
print(x_train.shape, y_train.shape)
print(x_valid.shape, y_valid.shape)
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, 100, 100)          10489900

 dropout (Dropout)           (None, 100, 100)          0

 conv1d (Conv1D)             (None, 100, 32)           16032

 max_pooling1d (MaxPooling1D  (None, 50, 32)           0
 )

 conv1d_1 (Conv1D)           (None, 50, 64)            6208

 max_pooling1d_1 (MaxPooling  (None, 25, 64)           0
```

```
 1D)

 lstm (LSTM)                 (None, 100)               66000

 batch_normalization (BatchN  (None, 100)              400
 ormalization)

 dense (Dense)               (None, 256)               25856

 dense_1 (Dense)             (None, 128)               32896

 dense_2 (Dense)             (None, 64)                8256

 dense_3 (Dense)             (None, 1)                 65

=================================================================
Total params: 10,645,613
Trainable params: 10,645,413
Non-trainable params: 200
_________________________________________________________________
None
Epoch 1/8
1312/1312 [==============================] - 247s 186ms/step - loss: 0.4158 - accurac
Epoch 2/8
1312/1312 [==============================] - 243s 185ms/step - loss: 0.2700 - accurac
Epoch 3/8
1312/1312 [==============================] - 242s 185ms/step - loss: 0.2246 - accurac
Epoch 4/8
1312/1312 [==============================] - 242s 184ms/step - loss: 0.1929 - accurac
Epoch 5/8
1312/1312 [==============================] - 244s 186ms/step - loss: 0.1694 - accurac
Epoch 6/8
1312/1312 [==============================] - 242s 184ms/step - loss: 0.1438 - accurac
Epoch 7/8
1312/1312 [==============================] - 243s 185ms/step - loss: 0.1252 - accurac
Epoch 8/8
1312/1312 [==============================] - 243s 186ms/step - loss: 0.0979 - accurac
(83918, 100) (83918,)
(20980, 100) (20980,)
```

```
with open("/content/drive/MyDrive/CNNfake","wb") as f:
  pickle.dump(modell,f)
```

```
WARNING:absl:Found untraced functions such as lstm_cell_1_layer_call_fn, lstm_cell_1_
INFO:tensorflow:Assets written to: ram://45055858-149c-4560-adac-4dd39ea86ce1/assets
INFO:tensorflow:Assets written to: ram://45055858-149c-4560-adac-4dd39ea86ce1/assets
WARNING:absl:<keras.layers.recurrent.LSTMCell object at 0x7fe136f24d10> has the same
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.m
```

```
with open("/content/drive/MyDrive/CNNfake","rb") as f:
  mp=pickle.load(f)
```

```
from urllib.request import urlopen
from bs4 import BeautifulSoup

url = "https://www.bbc.com/news/world-africa-58260855"
html = urlopen(url).read()
soup = BeautifulSoup(html, features="html.parser")

# kill all script and style elements
for script in soup(["script", "style"]):
  script.extract() # rip it out

# get text
text = soup.get_text()

# break into lines and remove leading and trailing space on each
lines = (line.strip() for line in text.splitlines())
# break multi-headlines into a line each
chunks = (phrase.strip() for line in lines for phrase in line.split(" "))
# drop blank lines
text = '\n'.join(chunk for chunk in chunks if chunk)

print(text)
```

```
import urllib.request
from bs4 import BeautifulSoup

# providing url
url = "https://www.bbc.com/news/world-africa-58260855"

# opening the url for reading
html = urllib.request.urlopen(url)

# parsing the html file
htmlParse = BeautifulSoup(html, 'html.parser')
arr=[]
# getting all the paragraphs
for para in htmlParse.find_all("p"):
    arr.append(para.get_text())
```

```
result=[]
for i in arr:
  #print(i)
  process=_preprocess(i)
  #print(process)
  y_predict = mp.predict(process)
  classes_x = (y_predict > 0.5).astype("int32")
  result.append(classes_x)
print(result)
```

```
print(np.count_nonzero(classes_x==1))
print(np.count_nonzero(classes_x==0))
```

```
text="29 Humans Killed in Laboratory Producing Militarized Artificial Intelligence Ro
process=_preprocess(text)
y_predict = mp.predict(process)
classes_x = (y_predict > 0.5).astype("int32")
```

```
real=np.count_nonzero(classes_x==1)
fake=np.count_nonzero(classes_x==0)
```

```
from statistics import mode
```

```
if(real>fake):
  print("Real")
else:
  print("Fake")
```

```
Real
```

```
1
```

✓ 0s completed at 3:19 PM